Software Manual Testing

1. Basics of Software Testing

Software Testing is the process of verifying and validating that a software application works as intended and meets user requirements — without defects. It helps ensure quality, reliability, and user satisfaction.

Goals of Testing:

• Detect defects early
•Ensure software quality
• Verify requirements
• Prevent issues in production
• Build confidence in product stability

2. Software Testing Levels

• Unit Testing – Testing individual units or components.
• Integration Testing – Testing interactions between integrated modules.
• System Testing – End-to-end testing of the entire application.
• Acceptance Testing (UAT) – Testing from end-user perspective.

3. Types of Software Testing

Testing is broadly categorized into
Functional and Non-Functional Testing. Functional testing verifies
what the system does; Non-Functional testing checks how well it
performs.

A. Functional Testing

Purpose: To verify that each feature performs as expected according to requirements.

• Smoke Testing – Check build stability.
• Sanity Testing – Quick testing after minor fixes.
• Regression Testing – Ensure new changes don’t break existing features.
• Retesting – Recheck fixed defects.
• Integration Testing – Test combined modules.
• System Testing – Test complete workflow.
• UAT – Validate real business flow.
• Interface Testing – Validate data between systems.
• Ad-hoc/Exploratory Testing – Random, unplanned testing.

Common Tools:

Manual Testing, Jira, TestLink, BrowserStack, Postman

B. Non-Functional Testing

Purpose: To verify system performance, scalability, usability, and security.

• Performance Testing – Measure speed and stability.
• Load Testing – Verify system behavior under expected load.
• Stress Testing – Push system beyond capacity.

• Security Testing – Identify vulnerabilities.
• Usability Testing – Evaluate ease of use.
• Compatibility Testing – Verify across OS/browsers/devices.
• Reliability Testing – Check stability over time.
• Accessibility Testing – Ensure usability by disabled users.
• Localization Testing – Validate app in multiple languages.

Common Tools:
JMeter, LoadRunner, OWASP ZAP, Burp Suite, BrowserStack, AXE, WAVE, Hotjar, Lokalise

4. Test Case Design Techniques
• Equivalence Partitioning – Divide input into valid/invalid sets.
• Boundary Value Analysis – Test boundary limits.
• Decision Table Testing – Test input combinations.
• State Transition Testing – Based on state changes.
• Use Case Testing – Derived from real user flows.

5. Bug Life Cycle
New → Assigned → Open → Fixed → Retest → Verified → Closed → Reopened → Deferred →Rejected

6. Sample Bug Report Format
Bug ID: BUG_105
Module: Login
Summary: Login button not working
Severity: High
Priority: P1
Steps: 1. Go to login
2. Enter credentials
3. Click login
Expected: User logs in
Actual: Nothing happens
Environment: Chrome/Windows 11
Reported By: Hiro
Status: New
Date: 07-Nov-2025

7. SDLC vs STLC
SDLC: Requirement → Design → Development → Testing → Deployment → Maintenance
STLC: Requirement Analysis → Test Planning → Test Case Design → Test Execution → Test Closure

8. Common Tools for Manual Testers
• Test Management: TestLink, Zephyr, TestRail
• Bug Tracking: Jira, Bugzilla, Trello
• API Testing: Postman, Swagger
• Performance Testing: JMeter, LoadRunner

• Browser Testing: BrowserStack, CrossBrowserTesting
• Security Testing: OWASP ZAP, Burp Suite
• Accessibility Testing: AXE, WAVE
•Documentation: Excel, Confluence

9. Important Test Artifacts
• Test Plan
• Test Scenarios
• Test Cases
• Test Data
• RTM (Requirement Traceability Matrix)
• Defect Report
•Test Summary Report

10. Common Interview Topics for Freshers
• Verification vs Validation
• Severity vs Priority
• Smoke vs Sanity
• Regression vs Retesting
• Positive vs Negative Testing
• Agile Testing
• Bug Life Cycle
• STLC vs SDLC
• Test Case Design Techniques
• Functional vs Non-Functional Testing

11. Best Practices for Manual Testers
• Understand requirements clearly.
• Test from end-user perspective.
• Write clear bug reports.
• Maintain screenshots/logs.
•Prioritize critical test cases.
• Communicate effectively.
•Learn SQL & API basics.
• Stay updated with automation and AI tools.

12. Artificial Intelligence
• AI.
• Generative AI.
• Prompt Engineering.
• LLM.
• AI agent.
• Chrome DevTools
•TestRigor

13. Practice Resources

- https://demo.opencart.com
- https://practicesoftwaretesting.com
- https://the-internet.herokuapp.com
- https://www.saucedemo.com/

Try writing test cases,
executing manually, and logging defects using Jira or TestLink.